

Data Science Homework

CSCI 62

Introduction

This homework has two parts. In the first part, you will work with a real data set to implement a classifier for Twitter Bots. In the second part, you will create an artificial dataset to better understand the difference between decision tree classifiers and linear classifiers

Installation. Download Jupyter notebook (see setup.pdf on Camino). If you have trouble installing Jupyter, you can also use Google Colab online (reading in from files is a little trickier. see this tutorial: <https://medium.com/@seema.singh/reading-and-writing-files-with-google-> I have also heard that you can use .ipynb notebooks on pycharm, so if that is the case you are welcome to use that as well.

Set up. Create a .ipynb notebook and download this file https://github.com/abacadaea/csci62/blob/main/hw6/clean_data.csv into the same folder.

Deliverable. An .ipynb file with your code, plots, and answers to questions.

1 Part 1: Twitter Bot Detection

1.1 Dataset Description

1. `id` - unique identifier for the user account.
2. `Screen_name` - public username or handle of the user.
3. `location` - a free-text field that allows the user to specify their physical location, such as a city, state, or country.
4. `description` - a brief bio or description of themselves or their account.
5. `url` - URL associated with the user account
6. `followers_count` - refers to the Twitter users that follow a specific user.
7. `friends_count` - Twitter users that a specific user follows
8. `listed_count` - A count of how many times the Tweet has been viewed

9. `created_at` - when was the profile created
10. `favourites_count` - number of tweets that the user has marked as favorites.
11. `verified` - True/False
12. `statuses_count` - total number of tweets that the user has posted.
13. `lang` - language that the user has specified for their account
14. `status` - the most recent tweet posted by the user
15. `default_profile` - boolean property that indicates whether the account has customized their profile, including their profile picture, header image, and bio
16. `default_profile_image` - boolean property that indicates whether the user has uploaded a profile picture or is using the default profile picture provided by Twitter.
17. `name` - full name of the user associated with the account
18. `bot` - target variable (0,1)

1.2 Python code

Step 1: Import all the libraries: `pandas`, `numpy`, `matplotlib`, `sklearn`.

Step 2: Read the training dataset into program

1. Use `pandas` to load the dataset into a `pandas` dataframe.
2. Print the first 5 rows of the dataframe along with the size of the dataframe.

Step 3: Data Processing

1. Find the number of missing values in each column of your dataframe.
2. Delete all the columns that have more than 35% of the values missing.

Step 4: Visualization: Friends vs Followers count

1. Use `Matplotlib` scatter plot to visualize `friends_count` vs `followers_counts` for both bots and nonbots. Include a legend in your plot.

Step 5: Feature Engineering

1. `Screen_name` is a feature in the dataset that describes the public username or handle of the user.

2. Using the following list of strings,

```
bag_of_words=[bot, follow me, free, fake, virus, troll, clone, tweet me]
```

introduce new columns

- `screen_name_binary`,
- `name_binary`
- `description_binary`
- `status_binary`

The values in each of these columns stores `True` when the value in the corresponding column contains a substring from the `bag_of_words` and `False` when it does not. The value should be `False` when the corresponding value is `NaN`. For example:

<code>screen_name</code>	<code>screen_name_binary</code>
mattlieberisbot	True
aaroncarter	False
NaN	False

Step 6: Decision Tree Classifier

1. Import `DecisionTreeClassifier`:

```
from sklearn.tree import DecisionTreeClassifier
```

2. Create a subset of the dataframe that includes the target variable as well as the following features:

- `friends_count`
- `followers_count`
- `screen_name_binary`
- `name_binary`
- `status_binary`
- `verified`
- `statuses_count`

3. Split the data from part 2 into X (features) and Y (Target variable)
4. Split the data from part 3 into 70% training and 30% testing dataset.

5. Implement the Decision Tree Classifier (refer to code below)

	Day	Outlook	Temperature	Humidity	Wind	Play_Tennis
0	0	2	1	0	1	0
1	6	2	1	0	0	0
2	7	0	1	0	1	1
3	8	1	2	0	1	1
4	9	1	0	1	1	1
5	10	1	0	1	0	0
6	11	0	0	1	0	1
7	12	2	2	0	1	0
8	13	2	0	1	1	1
9	1	1	2	1	1	1
10	2	2	2	1	0	1
11	3	0	2	0	0	1
12	4	0	1	1	1	1
13	5	1	2	0	0	0

```
#To divide our data into attribute set and Label:
feature_cols = ['Outlook','Temperature','Humidity','Wind']
X = df[feature_cols ]           #contains the attribute
y = df.Play_Tennis             #contains the Label
```

```
clf = tree.DecisionTreeClassifier(criterion='entropy')
clf.fit(X,y)
```

```
DecisionTreeClassifier(criterion='entropy')
```

```
y_pred= clf.predict(X)
from sklearn.metrics import accuracy_score
print("Accuracy:",metrics.accuracy_score(y, y_pred))
```

```
Accuracy: 1.0
```

6. Print the training and testing accuracy.
7. Using the method `sklearn.tree.export_text`, get a text based visualization of your classifier by printing the value of `export_text(clf, feature_names=list(X.columns))`.

Step 7: Linear Classifier

1. Import the `LogisticRegression` classifier:

```
from sklearn.linear_model import LogisticRegression
```

This is a classifier that implements a linear model (contrary to what the name suggests, a `LogisticRegression` classifier is actually used for classification).

2. Using the same training and test data, fit a `LogisticRegression` classifier. The code should be almost the same, except that you use `clf = LogisticRegression()` instead of `clf=tree.DecisionTreeClassifier()`. You still use the `.fit` and `.predict` methods in the same way.
3. Print the training and testing accuracy for the Logistic Regression classifier.

1.3 Short Answer

Questions to answer (Write at most a paragraph for each): You can add your answers to your `.ipynb` notebook, or you can submit a separate file.

- Based on the plot in step 4 part 1, can you determine if `friends_count` and `followers_count` can be used for classification? Is there a pattern? Consider zooming in on the plot to get a better picture.
- For the `DecisionTree`, what was the training accuracy? What was the testing accuracy? Which of the training and test accuracies better represent the quality of the classifier, and why?
- Which of the `DecisionTree` classifier and the `LogisticRegression` classifier is better at this classification task? Why do you think this is?

4. The `DecisionTree` classifier you printed out in Step 6 with `export_text(clf)` is quite complicated. A natural question is how well a simple decision tree might perform.
 - (a) Copy your code from Step 6 into a new cell. Relearn a `DecisionTree` classifier on the training data, but this time, initialize the classifier with the parameter `max_depth=2`. That is:

```
clf = tree.DecisionTreeClassifier(criterion='entropy', max_depth=2)
```
 - (b) After fitting the classifier, look at the decision tree by running `export_text(clf)` again. How does this decision tree classify users?
 - (c) What does this tell you about what the classifier think are the most important features for detecting whether a user is a bot?
5. What percentage of the training and test data are bots? If you had a different dataset where only 10% of the rows were bots, would 85% accuracy be good?
6. When you split the data into train and test in Step 6, you might have assigned the first 70% of the data to the training data and the last 30% of the data as test data. Can you see any problems with doing this? Can you think of any better alternatives?
7. Often in data science you are just given a data set and a task (e.g., learn a classifier for detecting bots). The instructions in the assignment made a lot of decisions for you. List 5 examples of these decisions that the assignment made for you. (In other words, if you were trying to solve this problem without this homework guiding you, what would you have to decide yourself in processing your data and designing your classifier?)

2 Part 2: More with Decision Tree and Linear Classifiers

Complete Part 2 in the same `.ipynb` file as Part 1. In this problem, you will look at an artificial dataset that highlights the difference between decision tree classifiers and linear classifiers.

2.1 Python code

Step 1: Create an artificial dataset Create a dataset, stored in a Pandas Dataframe, with the following properties:

1. The dataset has 2 numerical input variables named `X1` and `X2`.
2. The dataset has 1 binary output variable named `Y`, which is 0 or 1.
3. This dataset has 1000 rows, generated as follows: for each row, generate `X1` and `X2` as (independent) random real numbers between 0 and 1. The value of `Y` should be 1 if $X1+X2 > 1$ and 0 otherwise. To get random numbers between 0 and 1, you can `import random` and use `random.random()`.

Hint:

- Here is an example of creating a DataFrame from the X1, X2, and Y values.

```
X1s = [0,1,2]
X2s = [3,4,5]
Ys = [0,0,1]
# create dataset with points (0,3),(1,4),(2,5), with Y-labels 0,0,1
df = pd.DataFrame(data={'X1':X1s,'X2':X2s,'Y':Ys})
```

Step 2: Visualization Create a scatter plot of your data that has the input variables on the two axes. The data points should be one of two colors, depending on the value of Y.

Step 3: Classifier Fit a `DecisionTreeClassifier` with parameter `max_depth=2` to this dataset. Train it on the entire dataset (don't use `train_test_split`). Then fit a `LogisticRegression` classifier to this dataset, and print the accuracy of each classifier (again, on the whole dataset).

2.2 Short answer: (answer in the notebook)

1. In Step 1, explain why you expect the linear classifier (`LogisticRegression`) to get 100% accuracy on this dataset.
2. What is the accuracy of your decision tree classifier?
3. Create a new cell and fit another `DecisionTreeClassifier` WITHOUT the parameter `max_depth=2` (this allows the depth of the tree to be arbitrary). Now what is the accuracy of the `DecisionTreeClassifier`?
4. Can you explain why a `DecisionTreeClassifier` with an arbitrarily large depth can achieve a near-perfect accuracy on this dataset? What is this `DecisionTreeClassifier` doing? (it may help to look at `export_text(clf)`)

2.3 Extra credit (10 points), submit a separate pdf/txt file

If you would like to try the extra credit, please submit a separate pdf or txt file, in addition to your ipynb file.

Motivation. In general, for any dataset where a linear classifier performs well, it is also possible to find a decision tree classifier that essentially matches the performance of the linear classifier. You saw an example of this above. However, a linear classifier may still be preferable, largely because a decision tree classifier suffers from what is called the *curse of dimensionality*.

The problem (10 points of extra credit). Suppose $d \geq 101$ is an odd integer.¹ Consider a binary classification task with d input variables $x_1, x_2, \dots, x_d \in [0, 1]$, where the output variable Y is 1 if $x_1 + x_2 + \dots + x_d > d/2$, and $Y = 0$ otherwise. (This is the d -dimensional generalization of the example you generated above) Suppose we have a Decision Tree classifier that is correct all inputs that are “not too close to boundary.” More precisely, suppose

- all inputs $(x_1, \dots, x_d)$ satisfying $x_1 + x_2 + \dots + x_d \leq (d-1)/2$ are classified correctly as $Y = 0$
- all inputs $(x_1, \dots, x_d)$ satisfying $x_1 + x_2 + \dots + x_d \geq (d+1)/2$ are classified correctly as $Y = 1$.
- all other inputs can be classified as either $Y = 0$ or $Y = 1$.

Show that any Decision Tree classifier satisfying the above properties must have at least $\binom{d}{(d-1)/2}$ leaf nodes. (Hint: consider inputs $(x_1, \dots, x_d)$ where each x_i is either 0 or 1)

The punchline. A crude estimate gives that $\binom{d}{(d-1)/2}$ is roughly $\frac{2^d}{\sqrt{d}}$ when d is large, so this problem shows that the size of the Decision Tree classifier grows exponentially with the dimension of the input. This stands in stark contrast to the linear classifier $x_1 + x_2 + \dots + x_d > d/2$, which is much simpler and takes only $O(d)$ coefficients to specify.

¹we assume d is odd to make the problem simpler. A similar statement holds if d is even